

Practical No. 1

Definition : Write a program to demonstrate basic input and output operations in Python.

Aim/ Objective:

- To understand how to accept input from the user in Python.
- To display output using the print() function.
- To perform basic operations on user-provided data.
- To understand basic Python syntax for input and output.

Source Code:

Write a program to demonstrate basic input and output operations in Python.

```
name = input("Enter name : ")
age = int(input("Enter age : "))

print("My name is ",name)
print(f"I am {age} years old ")
```

Output:

```
Enter name : Harsh
Enter age : 21
My name is Harsh
I am 21 years old
```

Conclusion:

The program successfully demonstrates the basic input and output operations in Python. It shows how to accept data from the user using input(), convert input into appropriate datatypes, store values in variables, perform a basic arithmetic operation, and display the results using print()

Practical No:2

Definition: Write a program to illustrate the use of different data types and type casting.

Aim/ Objective:

- To understand different data types such as integer, float, string, and Boolean.
- To learn how to declare and use variables with different data types.
- To understand type casting and conversion of one data type into another.
- To perform operations using different data types.
- To understand the importance of type conversion in programming.

Source Code:

```
# program to illustrate the use of different data types and type casting
```

```
name = "Harsh"
```

```
age = 21
```

```
marks = 56.5
```

```
is_correct = True
```

```
print(type(name))
```

```
print(type(age))
```

```
print(type(marks))
```

```
print(type(is_correct))
```

```
#type castings
```

```
age = 21
```

```
age_float = float(age)
```

```
marks = 56.5
```

```
marks_int = int(marks)
```

```
print("Integer to Float:", age_float)
```

```
print("Float to Integer:", marks_int)
```

Output:

```
<class 'str'>
```

```
<class 'int'>
```

<class 'float'>

<class 'bool'>

Integer to Float: 21.0

Float to Integer: 56

Conclusion:

The program successfully demonstrates the use of different data types such as integer, float, character, and string. It also illustrates type casting, where one data type is converted into another. Thus, we understand how data types and type conversion are used in programming to perform operations correctly and efficiently.

Practical No:3

Definition: Write a program to perform arithmetic relational and logical operations using Python operators.

Aim/Objective:

- To understand the use of arithmetic operators such as **+**, **-**, *****, **/**, **%**, **//**, and ******.
- To understand and implement relational operators such as **>**, **<**, **>=**, **<=**, **==**, and **!=**.
- To understand the use of logical operators such as **and**, **or**, and **not**.
- To perform mathematical calculations and comparisons using Python.
- To understand how operators are used to evaluate expressions and conditions in Python.

Source Code:

```
a = 10
```

```
b = 5
```

```
print("Arithmetic Operations:")
```

```
print("Addition =", a + b)
```

```
print("Subtraction =", a - b)
```

```
print("Multiplication =", a * b)
```

```
print("Division =", a / b)
```

```
print("Modulus =", a % b)
```

```
print("\nRelational Operations:")
```

```
print("a == b :", a == b)
```

```
print("a != b :", a != b)
```

```
print("a > b :", a > b)
```

```
print("a < b :", a < b)
```

```
print("a >= b :", a >= b)
```

```
print("a <= b :", a <= b)
```

```
print("\nLogical Operations:")
```

```
print("(a > b) and (a < 20) :", (a > b) and (a < 20))
```

```
print("(a < b) or (a < 20) :", (a < b) or (a < 20))
```

```
print("not(a > b) :", not(a > b))
```

Output:

Arithmetic Operations:

Addition = 15

Subtraction = 5

Multiplication = 50

Division = 2.0

Modulus = 0

Relational Operations:

a == b : False

a != b : True

a > b : True

a < b : False

a >= b : True

a <= b : False

Logical Operations:

(a > b) and (a < 20) : True

(a < b) or (a < 20) : True

not(a > b) : False

Conclusion:

The program successfully demonstrates the use of arithmetic, relational, and logical operators in Python. It helps in understanding how mathematical calculations, comparisons, and logical conditions are performed using Python operators. Thus, the program provides a basic understanding of operators and their practical use in Python programming.

Practical No 4

Definition: Write a program to demonstrate string operations including slicing formatting and built-in string functions.

Aim/Objectives:

- To understand the concept and manipulation of strings in Python.
- To perform string slicing to extract specific parts of a string.
- To use string formatting for displaying text and values effectively.
- To implement various built-in string functions such as upper(), lower(), replace(), find(), and strip().
- To understand how string operations are used in practical Python programming.

Source Code:

```
string = 'Python'
print("String in uppercase:", string.upper())
print("String in lowercase:", string.lower())
print("String slicing:", string[0:2])
print("String in title case:", string.title())
print("Is the string alphabetic?:", string.isalpha())
print("Count of 'string':", string.count("string"))
print("Does the string end with 'n'?:", string.endswith("n"))
```

Output:

String in uppercase: PYTHON

String in lowercase: python

String slicing: Py

String in title case: Python

Is the string alphabetic?: True

Count of 'string': 0

Does the string end with 'n?': True

Conclusion:

The program successfully demonstrates various string operations in Python, including slicing, formatting, and built-in string functions. It helps in understanding how strings can be accessed, modified, formatted, and manipulated efficiently using Python. Thus, the program provides a basic understanding of string handling in Python programming.

Practical No 5

Definition: Write a program to create and manipulate lists using indexing slicing and list comprehensions.

Aim/Objectives:

- To understand the concept of lists in Python.
- To create and access list elements using indexing.
- To extract specific elements from a list using slicing.
- To use list comprehensions for creating and processing lists efficiently.
- To perform basic list manipulation and understand how lists are used in Python programming.

Source Code:

```
numbers = [10, 20, 30, 40, 50]
```

```
print('Original list: ',numbers)
```

```
print("First element:", numbers[0])
```

```
print("Last element:", numbers[-1])
```

```
print("First three elements:", numbers[0:3])
```

```
print("Last three elements:", numbers[2:])
```

```
numbers.append(60)

print("After adding 60:", numbers)

squares = [x * x for x in numbers]

print("Squares using list comprehension:", squares)
```

Output:

Original list: [10, 20, 30, 40, 50]
First element: 10
Last element: 50
First three elements: [10, 20, 30]
Last three elements: [30, 40, 50]
After adding 60: [10, 20, 30, 40, 50, 60]
Squares using list comprehension: [100, 400, 900, 1600, 2500, 3600]

Conclusion:

The program successfully demonstrates the creation and manipulation of lists in Python using indexing, slicing, and list comprehensions. It helps in understanding how to access, extract, and process list elements efficiently. Thus, the program provides a basic understanding of list operations and their practical use in Python.

Practical No 6

Definition: Write a program to illustrate the use of tuples and sets with basic operations.

Aim/Objective:

- To understand the concept and characteristics of tuples and sets in Python.
- To create and access elements of a tuple.
- To perform basic tuple operations such as indexing, slicing, and concatenation.
- To create and manipulate sets using operations such as union, intersection, and difference.
- To understand the difference between tuples and sets and their practical applications in Python.

Source Code:

```
numbers = (10, 20, 30, 40, 50)
print("Tuple:", numbers)
print("First element:", numbers[0])
print("Last element:", numbers[-1])
print("Tuple slicing:", numbers[1:4])
print("Length of tuple:", len(numbers))

# Set
A = {10, 20, 30, 40}
B = {30, 40, 50, 60}
print("\nSet A:", A)
print("Set B:", B)
print("Union:", A | B)
print("Intersection:", A & B)
print("Difference:", A - B)
A.add(70)
print("After adding 70:", A)
A.remove(20)
print("After removing 20:", A)
```

Output:

Tuple: (10, 20, 30, 40, 50)

First element: 10

Last element: 50

Tuple slicing: (20, 30, 40)

Length of tuple: 5

Set A: {40, 10, 20, 30}

Set B: {40, 50, 60, 30}

Union: {40, 10, 50, 20, 60, 30}

Intersection: {40, 30}

Difference: {10, 20}

After adding 70: {70, 40, 10, 20, 30}

After removing 20: {70, 40, 10, 30}

Conclusion:

The program successfully demonstrates the use of tuples and sets along with their basic operations. It helps in understanding how tuples are used for storing ordered and immutable data, while sets are used for storing unique elements and performing mathematical set operations. Thus, the program provides a basic understanding of tuples and sets in Python.

Practical No 7

Definition: Write a program to create a dictionary and demonstrate dictionary methods and iteration.

Aim/Objective:

- To understand the concept and structure of dictionaries in Python.
- To create a dictionary using key-value pairs.
- To access, add, update, and remove dictionary elements.
- To demonstrate built-in dictionary methods such as keys(), values(), items(), and get().
- To iterate through dictionary elements using loops.
- To understand the practical use of dictionaries for storing and managing data.

Source Code:

```
student = {  
    "name": "Harsh",  
    "age": 21,  
    "marks": 85  
}  
  
print("Student Dictionary:", student)  
print("Name:", student["name"])  
print("Age:", student["age"])  
print("Keys:", student.keys())  
print("Values:", student.values())  
print("Items:", student.items())  
student["city"] = "Ahmedabad"
```

```
print("After adding city:", student)
student["marks"] = 90
print("After updating marks:", student)
student.pop("age")
print("After removing age:", student)X
print("\nDictionary Iteration:")
for key, value in student.items():
    print(key, ":", value)
```

Output:

```
Student Dictionary: {'name': 'Harsh', 'age': 21, 'marks': 85}
Name: Harsh
Age: 21
Keys: dict_keys(['name', 'age', 'marks'])
Values: dict_values(['Harsh', 21, 85])
Items: dict_items([('name', 'Harsh'), ('age', 21), ('marks', 85)])
After adding city: {'name': 'Harsh', 'age': 21, 'marks': 85, 'city': 'Ahmedabad'}
After updating marks: {'name': 'Harsh', 'age': 21, 'marks': 90, 'city': 'Ahmedabad'}
After removing age: {'name': 'Harsh', 'marks': 90, 'city': 'Ahmedabad'}

Dictionary Iteration:
name : Harsh
marks : 90
city : Ahmedabad
```

Conclusion:

The program successfully demonstrates the creation and manipulation of a dictionary in Python using various dictionary methods and iteration. It helps in understanding how data can be stored as key-value pairs, accessed, modified, and traversed efficiently. Thus, the program provides a basic understanding of dictionary operations in Python

Practical No 8

Definition: Write a program to explain mutable and immutable objects in Python.

Aim/Objective:

- To understand the difference between mutable and immutable objects in Python.
- To identify common mutable objects such as lists, sets, and dictionaries.
- To identify common immutable objects such as integers, strings, tuples, and floats.
- To demonstrate how mutable objects can be modified after creation.
- To demonstrate that immutable objects cannot be changed after creation.
- To understand how mutability affects data manipulation in Python

Source Code:

```
# Mutable object - List
list1 = [10, 20, 30]
print("Original List:", list1)

list1[0] = 100
print("Modified List:", list1)

# Immutable object - Tuple
tuple1 = (10, 20, 30)
print("Original Tuple:", tuple1)

print("Tuple is immutable and cannot be changed.")
```

Output:

```
Original List: [10, 20, 30]
Modified List: [100, 20, 30]
Original Tuple: (10, 20, 30)
Tuple is immutable and cannot be changed.
```

Conclusion:

The program demonstrates the difference between mutable and immutable objects in Python. Immutable objects cannot be changed after creation, whereas mutable objects can be

modified. This helps in understanding how Python handles data and memory for different types of objects.

Practical No:9

Definition: Write a program to define and use user-defined functions with different types of arguments.

Aim/Objectives:

- To understand the concept of user-defined functions in Python.
- To learn how to define and call functions.
- To demonstrate different types of function arguments.
- To understand the use of positional, keyword, and default arguments.
- To improve code reusability and organization.

Source Code:

```
def add(a, b):  
    print("Addition:", a + b)  
  
def greet(name="Student"):  
    print("Hello", name)  
  
def student(name, age):  
    print("Name:", name)  
    print("Age:", age)  
  
add(10, 20)  
greet()  
student(age=20, name="Harsh")
```

Output:

Addition: 30

Hello Student

Name: Harsh

Age: 20

Practical No 10

Definition: Write a program to demonstrate recursion using factorial or Fibonacci series.

Aim/Objective:

- To understand the concept of recursion in Python.
- To learn how a function can call itself.
- To demonstrate recursion using factorial or Fibonacci series.
- To understand the **base condition** required to stop recursion.
- To develop problem-solving skills using recursive functions.

Source Code:

```
def factorial(n):  
    if n == 0 or n == 1:  
        return 1  
    else:  
        return n * factorial(n - 1)  
num = int(input("Enter a number: "))  
print("Factorial =", factorial(num))
```

Output:

Enter a number: 9

Factorial = 362880